

If we let

$$\begin{aligned}
 x &= \frac{kq}{(n-k)p} \\
 &< \frac{kq}{(n-np)p} \\
 &= \frac{kq}{nqp} \\
 &= \frac{k}{np} \\
 &< 1,
 \end{aligned}$$

it follows that

$$b(i-1; n, p) < x b(i; n, p)$$

for $0 < i \leq k$. Iteratively applying this inequality $k-i$ times, we obtain

$$b(i; n, p) < x^{k-i} b(k; n, p)$$

for $0 \leq i < k$, and hence

$$\begin{aligned}
 \sum_{i=0}^{k-1} b(i; n, p) &< \sum_{i=0}^{k-1} x^{k-i} b(k; n, p) \\
 &< b(k; n, p) \sum_{i=1}^{\infty} x^i \\
 &= \frac{x}{1-x} b(k; n, p) \\
 &= \frac{kq}{np-k} b(k; n, p). \quad \blacksquare
 \end{aligned}$$

Corollary C.5

Consider a sequence of n Bernoulli trials, where success occurs with probability p and failure with probability $q = 1 - p$. Then for $0 < k \leq np/2$, the probability of fewer than k successes is less than one half of the probability of fewer than $k + 1$ successes.

Proof Because $k \leq np/2$, we have

$$\frac{kq}{np-k} \leq \frac{(np/2)q}{np-(np/2)}$$

$$\begin{aligned}
&= \frac{(np/2)q}{np/2} \\
&\leq 1,
\end{aligned} \tag{C.42}$$

since $q \leq 1$. Letting X be the random variable denoting the number of successes, Theorem C.4 and inequality (C.42) imply that the probability of fewer than k successes is

$$\Pr\{X < k\} = \sum_{i=0}^{k-1} b(i; n, p) < b(k; n, p).$$

Thus we have

$$\begin{aligned}
\frac{\Pr\{X < k\}}{\Pr\{X < k+1\}} &= \frac{\sum_{i=0}^{k-1} b(i; n, p)}{\sum_{i=0}^k b(i; n, p)} \\
&= \frac{\sum_{i=0}^{k-1} b(i; n, p)}{\sum_{i=0}^{k-1} b(i; n, p) + b(k; n, p)} \\
&< 1/2,
\end{aligned}$$

since $\sum_{i=0}^{k-1} b(i; n, p) < b(k; n, p)$. ■

Bounds on the right tail follow similarly. Exercise C.5-2 asks you to prove them.

Corollary C.6

Consider a sequence of n Bernoulli trials, where success occurs with probability p . Let X be the random variable denoting the total number of successes. Then for $np < k < n$, the probability of more than k successes is

$$\begin{aligned}
\Pr\{X > k\} &= \sum_{i=k+1}^n b(i; n, p) \\
&< \frac{(n-k)p}{k-np} b(k; n, p).
\end{aligned} \quad \blacksquare$$

Corollary C.7

Consider a sequence of n Bernoulli trials, where success occurs with probability p and failure with probability $q = 1 - p$. Then for $(np + n)/2 < k < n$, the probability of more than k successes is less than one half of the probability of more than $k - 1$ successes. ■

The next theorem considers n Bernoulli trials, each with a probability p_i of success, for $i = 1, 2, \dots, n$. As the subsequent corollary shows, we can use the

theorem to provide a bound on the right tail of the binomial distribution by setting $p_i = p$ for each trial.

Theorem C.8

Consider a sequence of n Bernoulli trials, where in the i th trial, for $i = 1, 2, \dots, n$, success occurs with probability p_i and failure occurs with probability $q_i = 1 - p_i$. Let X be the random variable describing the total number of successes, and let $\mu = E[X]$. Then for $r > \mu$,

$$\Pr\{X - \mu \geq r\} \leq \left(\frac{\mu e}{r}\right)^r.$$

Proof Since for any $\alpha > 0$, the function $e^{\alpha x}$ is strictly increasing in x ,

$$\Pr\{X - \mu \geq r\} = \Pr\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\}, \quad (\text{C.43})$$

where we will determine α later. Using Markov's inequality (C.30), we obtain

$$\Pr\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\} \leq E[e^{\alpha(X-\mu)}] e^{-\alpha r}. \quad (\text{C.44})$$

The bulk of the proof consists of bounding $E[e^{\alpha(X-\mu)}]$ and substituting a suitable value for α in inequality (C.44). First, we evaluate $E[e^{\alpha(X-\mu)}]$. Using the technique of indicator random variables (see Section 5.2), let $X_i = I\{\text{the } i\text{th Bernoulli trial is a success}\}$ for $i = 1, 2, \dots, n$; that is, X_i is the random variable that is 1 if the i th Bernoulli trial is a success and 0 if it is a failure. Thus,

$$X = \sum_{i=1}^n X_i,$$

and by linearity of expectation,

$$\mu = E[X] = E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n p_i,$$

which implies

$$X - \mu = \sum_{i=1}^n (X_i - p_i).$$

To evaluate $E[e^{\alpha(X-\mu)}]$, we substitute for $X - \mu$, obtaining

$$\begin{aligned} E[e^{\alpha(X-\mu)}] &= E[e^{\alpha \sum_{i=1}^n (X_i - p_i)}] \\ &= E\left[\prod_{i=1}^n e^{\alpha(X_i - p_i)}\right] \\ &= \prod_{i=1}^n E[e^{\alpha(X_i - p_i)}], \end{aligned}$$

which follows from (C.24), since the mutual independence of the random variables X_i implies the mutual independence of the random variables $e^{\alpha(X_i - p_i)}$ (see Exercise C.3-5). By the definition of expectation,

$$\begin{aligned}
 \mathbb{E}[e^{\alpha(X_i - p_i)}] &= e^{\alpha(1-p_i)} p_i + e^{\alpha(0-p_i)} q_i \\
 &= p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \\
 &\leq p_i e^{\alpha} + 1 \\
 &\leq \exp(p_i e^{\alpha}),
 \end{aligned} \tag{C.45}$$

where $\exp(x)$ denotes the exponential function: $\exp(x) = e^x$. (Inequality (C.45) follows from the inequalities $\alpha > 0$, $q_i \leq 1$, $e^{\alpha q_i} \leq e^{\alpha}$, and $e^{-\alpha p_i} \leq 1$, and the last line follows from inequality (3.12).) Consequently,

$$\begin{aligned}
 \mathbb{E}[e^{\alpha(X - \mu)}] &= \prod_{i=1}^n \mathbb{E}[e^{\alpha(X_i - p_i)}] \\
 &\leq \prod_{i=1}^n \exp(p_i e^{\alpha}) \\
 &= \exp\left(\sum_{i=1}^n p_i e^{\alpha}\right) \\
 &= \exp(\mu e^{\alpha}),
 \end{aligned} \tag{C.46}$$

since $\mu = \sum_{i=1}^n p_i$. Therefore, from equation (C.43) and inequalities (C.44) and (C.46), it follows that

$$\Pr\{X - \mu \geq r\} \leq \exp(\mu e^{\alpha} - \alpha r). \tag{C.47}$$

Choosing $\alpha = \ln(r/\mu)$ (see Exercise C.5-7), we obtain

$$\begin{aligned}
 \Pr\{X - \mu \geq r\} &\leq \exp(\mu e^{\ln(r/\mu)} - r \ln(r/\mu)) \\
 &= \exp(r - r \ln(r/\mu)) \\
 &= \frac{e^r}{(r/\mu)^r} \\
 &= \left(\frac{\mu e}{r}\right)^r.
 \end{aligned}$$

■

When applied to Bernoulli trials in which each trial has the same probability of success, Theorem C.8 yields the following corollary bounding the right tail of a binomial distribution.

Corollary C.9

Consider a sequence of n Bernoulli trials, where in each trial success occurs with probability p and failure occurs with probability $q = 1 - p$. Then for $r > np$,

$$\begin{aligned} \Pr\{X - np \geq r\} &= \sum_{k=\lceil np+r \rceil}^n b(k; n, p) \\ &\leq \left(\frac{npe}{r}\right)^r. \end{aligned}$$

Proof By equation (C.37), we have $\mu = E[X] = np$. ■

Exercises**C.5-1 ★**

Which is less likely: obtaining no heads when you flip a fair coin n times, or obtaining fewer than n heads when you flip the coin $4n$ times?

C.5-2 ★

Prove Corollaries C.6 and C.7.

C.5-3 ★

Show that

$$\sum_{i=0}^{k-1} \binom{n}{i} a^i < (a+1)^n \frac{k}{na - k(a+1)} b(k; n, a/(a+1))$$

for all $a > 0$ and all k such that $0 < k < na/(a+1)$.

C.5-4 ★

Prove that if $0 < k < np$, where $0 < p < 1$ and $q = 1 - p$, then

$$\sum_{i=0}^{k-1} p^i q^{n-i} < \frac{kq}{np - k} \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.$$

C.5-5 ★

Use Theorem C.8 to show that

$$\Pr\{\mu - X \geq r\} \leq \left(\frac{(n - \mu)e}{r}\right)^r$$

for $r > n - \mu$. Similarly, use Corollary C.9 to show that

$$\Pr\{np - X \geq r\} \leq \left(\frac{nqe}{r}\right)^r$$

for $r > n - np$.

C.5-6 ★

Consider a sequence of n Bernoulli trials, where in the i th trial, for $i = 1, 2, \dots, n$, success occurs with probability p_i and failure occurs with probability $q_i = 1 - p_i$. Let X be the random variable describing the total number of successes, and let $\mu = E[X]$. Show that for $r \geq 0$,

$$\Pr\{X - \mu \geq r\} \leq e^{-r^2/2n}.$$

(Hint: Prove that $p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \leq e^{\alpha^2/2}$. Then follow the outline of the proof of Theorem C.8, using this inequality in place of inequality (C.45).)

C.5-7 ★

Show that choosing $\alpha = \ln(r/\mu)$ minimizes the right-hand side of inequality (C.47).

Problems
C-1 Balls and bins

In this problem, we investigate the effect of various assumptions on the number of ways of placing n balls into b distinct bins.

- a. Suppose that the n balls are distinct and that their order within a bin does not matter. Argue that the number of ways of placing the balls in the bins is b^n .
- b. Suppose that the balls are distinct and that the balls in each bin are ordered. Prove that there are exactly $(b+n-1)!/(b-1)!$ ways to place the balls in the bins. (Hint: Consider the number of ways of arranging n distinct balls and $b-1$ indistinguishable sticks in a row.)
- c. Suppose that the balls are identical, and hence their order within a bin does not matter. Show that the number of ways of placing the balls in the bins is $\binom{b+n-1}{n}$. (Hint: Of the arrangements in part (b), how many are repeated if the balls are made identical?)
- d. Suppose that the balls are identical and that no bin may contain more than one ball, so that $n \leq b$. Show that the number of ways of placing the balls is $\binom{b}{n}$.
- e. Suppose that the balls are identical and that no bin may be left empty. Assuming that $n \geq b$, show that the number of ways of placing the balls is $\binom{n-1}{b-1}$.

Appendix notes

The first general methods for solving probability problems were discussed in a famous correspondence between B. Pascal and P. de Fermat, which began in 1654, and in a book by C. Huygens in 1657. Rigorous probability theory began with the work of J. Bernoulli in 1713 and A. De Moivre in 1730. Further developments of the theory were provided by P.-S. Laplace, S.-D. Poisson, and C. F. Gauss.

Sums of random variables were originally studied by P. L. Chebyshev and A. A. Markov. A. N. Kolmogorov axiomatized probability theory in 1933. Chernoff [66] and Hoeffding [173] provided bounds on the tails of distributions. Seminal work in random combinatorial structures was done by P. Erdős.

Knuth [209] and Liu [237] are good references for elementary combinatorics and counting. Standard textbooks such as Billingsley [46], Chung [67], Drake [95], Feller [104], and Rozanov [300] offer comprehensive introductions to probability.

D Matrices

Matrices arise in numerous applications, including, but by no means limited to, scientific computing. If you have seen matrices before, much of the material in this appendix will be familiar to you, but some of it might be new. Section D.1 covers basic matrix definitions and operations, and Section D.2 presents some basic matrix properties.

D.1 Matrices and matrix operations

In this section, we review some basic concepts of matrix theory and some fundamental properties of matrices.

Matrices and vectors

A **matrix** is a rectangular array of numbers. For example,

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \end{aligned} \tag{D.1}$$

is a 2×3 matrix $A = (a_{ij})$, where for $i = 1, 2$ and $j = 1, 2, 3$, we denote the element of the matrix in row i and column j by a_{ij} . We use uppercase letters to denote matrices and corresponding subscripted lowercase letters to denote their elements. We denote the set of all $m \times n$ matrices with real-valued entries by $\mathbb{R}^{m \times n}$ and, in general, the set of $m \times n$ matrices with entries drawn from a set S by $S^{m \times n}$.

The **transpose** of a matrix A is the matrix A^T obtained by exchanging the rows and columns of A . For the matrix A of equation (D.1),

$$A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}.$$

A **vector** is a one-dimensional array of numbers. For example,

$$x = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

is a vector of size 3. We sometimes call a vector of length n an **n -vector**. We use lowercase letters to denote vectors, and we denote the i th element of a size- n vector x by x_i , for $i = 1, 2, \dots, n$. We take the standard form of a vector to be as a **column vector** equivalent to an $n \times 1$ matrix; the corresponding **row vector** is obtained by taking the transpose:

$$x^T = (2 \ 3 \ 5).$$

The **unit vector** e_i is the vector whose i th element is 1 and all of whose other elements are 0. Usually, the size of a unit vector is clear from the context.

A **zero matrix** is a matrix all of whose entries are 0. Such a matrix is often denoted 0, since the ambiguity between the number 0 and a matrix of 0s is usually easily resolved from context. If a matrix of 0s is intended, then the size of the matrix also needs to be derived from the context.

Square matrices

Square $n \times n$ matrices arise frequently. Several special cases of square matrices are of particular interest:

1. A **diagonal matrix** has $a_{ij} = 0$ whenever $i \neq j$. Because all of the off-diagonal elements are zero, we can specify the matrix by listing the elements along the diagonal:

$$\text{diag}(a_{11}, a_{22}, \dots, a_{nn}) = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{pmatrix}.$$

2. The $n \times n$ **identity matrix** I_n is a diagonal matrix with 1s along the diagonal:

$$\begin{aligned} I_n &= \text{diag}(1, 1, \dots, 1) \\ &= \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}. \end{aligned}$$

When I appears without a subscript, we derive its size from the context. The i th column of an identity matrix is the unit vector e_i .

3. A **tridiagonal matrix** T is one for which $t_{ij} = 0$ if $|i - j| > 1$. Nonzero entries appear only on the main diagonal, immediately above the main diagonal ($t_{i,i+1}$ for $i = 1, 2, \dots, n - 1$), or immediately below the main diagonal ($t_{i+1,i}$ for $i = 1, 2, \dots, n - 1$):

$$T = \begin{pmatrix} t_{11} & t_{12} & 0 & 0 & \dots & 0 & 0 & 0 \\ t_{21} & t_{22} & t_{23} & 0 & \dots & 0 & 0 & 0 \\ 0 & t_{32} & t_{33} & t_{34} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & t_{n-2,n-2} & t_{n-2,n-1} & 0 \\ 0 & 0 & 0 & 0 & \dots & t_{n-1,n-2} & t_{n-1,n-1} & t_{n-1,n} \\ 0 & 0 & 0 & 0 & \dots & 0 & t_{n,n-1} & t_{nn} \end{pmatrix}.$$

4. An **upper-triangular matrix** U is one for which $u_{ij} = 0$ if $i > j$. All entries below the diagonal are zero:

$$U = \begin{pmatrix} u_{11} & u_{12} & \dots & u_{1n} \\ 0 & u_{22} & \dots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & u_{nn} \end{pmatrix}.$$

An upper-triangular matrix is **unit upper-triangular** if it has all 1s along the diagonal.

5. A **lower-triangular matrix** L is one for which $l_{ij} = 0$ if $i < j$. All entries above the diagonal are zero:

$$L = \begin{pmatrix} l_{11} & 0 & \dots & 0 \\ l_{21} & l_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & l_{nn} \end{pmatrix}.$$

A lower-triangular matrix is **unit lower-triangular** if it has all 1s along the diagonal.

6. A **permutation matrix** P has exactly one 1 in each row or column, and 0s elsewhere. An example of a permutation matrix is

$$P = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

Such a matrix is called a permutation matrix because multiplying a vector x by a permutation matrix has the effect of permuting (rearranging) the elements of x . Exercise D.1-4 explores additional properties of permutation matrices.

7. A **symmetric matrix** A satisfies the condition $A = A^T$. For example,

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 6 & 4 \\ 3 & 4 & 5 \end{pmatrix}$$

is a symmetric matrix.

Basic matrix operations

The elements of a matrix or vector are numbers from a number system, such as the real numbers, the complex numbers, or integers modulo a prime. The number system defines how to add and multiply numbers. We can extend these definitions to encompass addition and multiplication of matrices.

We define **matrix addition** as follows. If $A = (a_{ij})$ and $B = (b_{ij})$ are $m \times n$ matrices, then their matrix sum $C = (c_{ij}) = A + B$ is the $m \times n$ matrix defined by

$$c_{ij} = a_{ij} + b_{ij}$$

for $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, n$. That is, matrix addition is performed componentwise. A zero matrix is the identity for matrix addition:

$$A + 0 = A = 0 + A.$$

If λ is a number and $A = (a_{ij})$ is a matrix, then $\lambda A = (\lambda a_{ij})$ is the **scalar multiple** of A obtained by multiplying each of its elements by λ . As a special case, we define the **negative** of a matrix $A = (a_{ij})$ to be $-1 \cdot A = -A$, so that the ij th entry of $-A$ is $-a_{ij}$. Thus,

$$A + (-A) = 0 = (-A) + A.$$

We use the negative of a matrix to define **matrix subtraction**: $A - B = A + (-B)$.

We define **matrix multiplication** as follows. We start with two matrices A and B that are **compatible** in the sense that the number of columns of A equals the number of rows of B . (In general, an expression containing a matrix product AB is always assumed to imply that matrices A and B are compatible.) If $A = (a_{ik})$ is an $m \times n$ matrix and $B = (b_{kj})$ is an $n \times p$ matrix, then their matrix product $C = AB$ is the $m \times p$ matrix $C = (c_{ij})$, where

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad (\text{D.2})$$

for $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, p$. The procedure SQUARE-MATRIX-MULTIPLY in Section 4.2 implements matrix multiplication in the straightforward manner based on equation (D.2), assuming that the matrices are square: $m = n = p$. To multiply $n \times n$ matrices, SQUARE-MATRIX-MULTIPLY performs n^3 multiplications and $n^2(n-1)$ additions, and so its running time is $\Theta(n^3)$.

Matrices have many (but not all) of the algebraic properties typical of numbers. Identity matrices are identities for matrix multiplication:

$$I_m A = A I_n = A$$

for any $m \times n$ matrix A . Multiplying by a zero matrix gives a zero matrix:

$$A 0 = 0.$$

Matrix multiplication is associative:

$$A(BC) = (AB)C$$

for compatible matrices A , B , and C . Matrix multiplication distributes over addition:

$$A(B + C) = AB + AC,$$

$$(B + C)D = BD + CD.$$

For $n > 1$, multiplication of $n \times n$ matrices is not commutative. For example, if

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \text{ and } B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \text{ then}$$

$$AB = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

and

$$BA = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

We define matrix-vector products or vector-vector products as if the vector were the equivalent $n \times 1$ matrix (or a $1 \times n$ matrix, in the case of a row vector). Thus, if A is an $m \times n$ matrix and x is an n -vector, then Ax is an m -vector. If x and y are n -vectors, then

$$x^T y = \sum_{i=1}^n x_i y_i$$

is a number (actually a 1×1 matrix) called the **inner product** of x and y . The matrix xy^T is an $n \times n$ matrix Z called the **outer product** of x and y , with $z_{ij} = x_i y_j$. The (**euclidean**) **norm** $\|x\|$ of an n -vector x is defined by

$$\begin{aligned} \|x\| &= (x_1^2 + x_2^2 + \cdots + x_n^2)^{1/2} \\ &= (x^T x)^{1/2}. \end{aligned}$$

Thus, the norm of x is its length in n -dimensional euclidean space.

Exercises

D.1-1

Show that if A and B are symmetric $n \times n$ matrices, then so are $A + B$ and $A - B$.

D.1-2

Prove that $(AB)^T = B^T A^T$ and that $A^T A$ is always a symmetric matrix.

D.1-3

Prove that the product of two lower-triangular matrices is lower-triangular.

D.1-4

Prove that if P is an $n \times n$ permutation matrix and A is an $n \times n$ matrix, then the matrix product PA is A with its rows permuted, and the matrix product AP is A with its columns permuted. Prove that the product of two permutation matrices is a permutation matrix.

D.2 Basic matrix properties

In this section, we define some basic properties pertaining to matrices: inverses, linear dependence and independence, rank, and determinants. We also define the class of positive-definite matrices.

Matrix inverses, ranks, and determinants

We define the **inverse** of an $n \times n$ matrix A to be the $n \times n$ matrix, denoted A^{-1} (if it exists), such that $AA^{-1} = I_n = A^{-1}A$. For example,

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}.$$

Many nonzero $n \times n$ matrices do not have inverses. A matrix without an inverse is called **noninvertible**, or **singular**. An example of a nonzero singular matrix is

$$\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}.$$

If a matrix has an inverse, it is called **invertible**, or **nonsingular**. Matrix inverses, when they exist, are unique. (See Exercise D.2-1.) If A and B are nonsingular $n \times n$ matrices, then

$$(BA)^{-1} = A^{-1}B^{-1}.$$

The inverse operation commutes with the transpose operation:

$$(A^{-1})^T = (A^T)^{-1}.$$

The vectors $x_1, x_2, \dots, x_n$ are **linearly dependent** if there exist coefficients $c_1, c_2, \dots, c_n$, not all of which are zero, such that $c_1x_1 + c_2x_2 + \dots + c_nx_n = 0$. The row vectors $x_1 = (1 \ 2 \ 3)$, $x_2 = (2 \ 6 \ 4)$, and $x_3 = (4 \ 11 \ 9)$ are linearly dependent, for example, since $2x_1 + 3x_2 - 2x_3 = 0$. If vectors are not linearly dependent, they are **linearly independent**. For example, the columns of an identity matrix are linearly independent.

The **column rank** of a nonzero $m \times n$ matrix A is the size of the largest set of linearly independent columns of A . Similarly, the **row rank** of A is the size of the largest set of linearly independent rows of A . A fundamental property of any matrix A is that its row rank always equals its column rank, so that we can simply refer to the **rank** of A . The rank of an $m \times n$ matrix is an integer between 0 and $\min(m, n)$, inclusive. (The rank of a zero matrix is 0, and the rank of an $n \times n$ identity matrix is n .) An alternate, but equivalent and often more useful, definition is that the rank of a nonzero $m \times n$ matrix A is the smallest number r such that there exist matrices B and C of respective sizes $m \times r$ and $r \times n$ such that

$$A = BC.$$

A square $n \times n$ matrix has **full rank** if its rank is n . An $m \times n$ matrix has **full column rank** if its rank is n . The following theorem gives a fundamental property of ranks.

Theorem D.1

A square matrix has full rank if and only if it is nonsingular. ■

A **null vector** for a matrix A is a nonzero vector x such that $Ax = 0$. The following theorem (whose proof is left as Exercise D.2-7) and its corollary relate the notions of column rank and singularity to null vectors.

Theorem D.2

A matrix A has full column rank if and only if it does not have a null vector. ■

Corollary D.3

A square matrix A is singular if and only if it has a null vector. ■

The ij th **minor** of an $n \times n$ matrix A , for $n > 1$, is the $(n-1) \times (n-1)$ matrix $A_{[ij]}$ obtained by deleting the i th row and j th column of A . We define the **determinant** of an $n \times n$ matrix A recursively in terms of its minors by

$$\det(A) = \begin{cases} a_{11} & \text{if } n = 1, \\ \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{[1j]}) & \text{if } n > 1. \end{cases}$$

The term $(-1)^{i+j} \det(A_{[ij]})$ is known as the **cofactor** of the element a_{ij} .

The following theorems, whose proofs are omitted here, express fundamental properties of the determinant.

Theorem D.4 (Determinant properties)

The determinant of a square matrix A has the following properties:

- If any row or any column of A is zero, then $\det(A) = 0$.
- The determinant of A is multiplied by λ if the entries of any one row (or any one column) of A are all multiplied by λ .
- The determinant of A is unchanged if the entries in one row (respectively, column) are added to those in another row (respectively, column).
- The determinant of A equals the determinant of A^T .
- The determinant of A is multiplied by -1 if any two rows (or any two columns) are exchanged.

Also, for any square matrices A and B , we have $\det(AB) = \det(A) \det(B)$. ■

Theorem D.5

An $n \times n$ matrix A is singular if and only if $\det(A) = 0$. ■

Positive-definite matrices

Positive-definite matrices play an important role in many applications. An $n \times n$ matrix A is **positive-definite** if $x^T A x > 0$ for all n -vectors $x \neq 0$. For example, the identity matrix is positive-definite, since for any nonzero vector $x = (x_1 \ x_2 \ \cdots \ x_n)^T$,

$$\begin{aligned} x^T I_n x &= x^T x \\ &= \sum_{i=1}^n x_i^2 \\ &> 0. \end{aligned}$$

Matrices that arise in applications are often positive-definite due to the following theorem.

Theorem D.6

For any matrix A with full column rank, the matrix $A^T A$ is positive-definite.

Proof We must show that $x^T (A^T A) x > 0$ for any nonzero vector x . For any vector x ,

$$\begin{aligned} x^T (A^T A) x &= (Ax)^T (Ax) \quad (\text{by Exercise D.1-2}) \\ &= \|Ax\|^2. \end{aligned}$$

Note that $\|Ax\|^2$ is just the sum of the squares of the elements of the vector Ax . Therefore, $\|Ax\|^2 \geq 0$. If $\|Ax\|^2 = 0$, every element of Ax is 0, which is to say $Ax = 0$. Since A has full column rank, $Ax = 0$ implies $x = 0$, by Theorem D.2. Hence, $A^T A$ is positive-definite. ■

Section 28.3 explores other properties of positive-definite matrices.

Exercises**D.2-1**

Prove that matrix inverses are unique, that is, if B and C are inverses of A , then $B = C$.

D.2-2

Prove that the determinant of a lower-triangular or upper-triangular matrix is equal to the product of its diagonal elements. Prove that the inverse of a lower-triangular matrix, if it exists, is lower-triangular.

D.2-3

Prove that if P is a permutation matrix, then P is invertible, its inverse is P^T , and P^T is a permutation matrix.

D.2-4

Let A and B be $n \times n$ matrices such that $AB = I$. Prove that if A' is obtained from A by adding row j into row i , then subtracting column i from column j of B yields the inverse B' of A' .

D.2-5

Let A be a nonsingular $n \times n$ matrix with complex entries. Show that every entry of A^{-1} is real if and only if every entry of A is real.

D.2-6

Show that if A is a nonsingular, symmetric, $n \times n$ matrix, then A^{-1} is symmetric. Show that if B is an arbitrary $m \times n$ matrix, then the $m \times m$ matrix given by the product BAB^T is symmetric.

D.2-7

Prove Theorem D.2. That is, show that a matrix A has full column rank if and only if $Ax = 0$ implies $x = 0$. (*Hint:* Express the linear dependence of one column on the others as a matrix-vector equation.)

D.2-8

Prove that for any two compatible matrices A and B ,

$$\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B)) ,$$

where equality holds if either A or B is a nonsingular square matrix. (*Hint:* Use the alternate definition of the rank of a matrix.)

Problems

D-1 Vandermonde matrix

Given numbers $x_0, x_1, \dots, x_{n-1}$, prove that the determinant of the *Vandermonde matrix*

$$V(x_0, x_1, \dots, x_{n-1}) = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix}$$

is

$$\det(V(x_0, x_1, \dots, x_{n-1})) = \prod_{0 \leq j < k \leq n-1} (x_k - x_j) .$$

(Hint: Multiply column i by $-x_0$ and add it to column $i + 1$ for $i = n - 1, n - 2, \dots, 1$, and then use induction.)

D-2 Permutations defined by matrix-vector multiplication over $GF(2)$

One class of permutations of the integers in the set $S_n = \{0, 1, 2, \dots, 2^n - 1\}$ is defined by matrix multiplication over $GF(2)$. For each integer x in S_n , we view its binary representation as an n -bit vector

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{pmatrix} ,$$

where $x = \sum_{i=0}^{n-1} x_i 2^i$. If A is an $n \times n$ matrix in which each entry is either 0 or 1, then we can define a permutation mapping each value x in S_n to the number whose binary representation is the matrix-vector product Ax . Here, we perform all arithmetic over $GF(2)$: all values are either 0 or 1, and with one exception the usual rules of addition and multiplication apply. The exception is that $1 + 1 = 0$. You can think of arithmetic over $GF(2)$ as being just like regular integer arithmetic, except that you use only the least significant bit.

As an example, for $S_2 = \{0, 1, 2, 3\}$, the matrix

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

defines the following permutation π_A : $\pi_A(0) = 0$, $\pi_A(1) = 3$, $\pi_A(2) = 2$, $\pi_A(3) = 1$. To see why $\pi_A(3) = 1$, observe that, working in $GF(2)$,

$$\begin{aligned} \pi_A(3) &= \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \cdot 1 + 0 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} , \end{aligned}$$

which is the binary representation of 1.

For the remainder of this problem, we work over $GF(2)$, and all matrix and vector entries are 0 or 1. We define the rank of a 0-1 matrix (a matrix for which each entry is either 0 or 1) over $GF(2)$ the same as for a regular matrix, but with all arithmetic that determines linear independence performed over $GF(2)$. We define the **range** of an $n \times n$ 0-1 matrix A by

$$R(A) = \{y : y = Ax \text{ for some } x \in S_n\} ,$$

so that $R(A)$ is the set of numbers in S_n that we can produce by multiplying each value x in S_n by A .

- a. If r is the rank of matrix A , prove that $|R(A)| = 2^r$. Conclude that A defines a permutation on S_n only if A has full rank.

For a given $n \times n$ matrix A and a given value $y \in R(A)$, we define the **preimage** of y by

$$P(A, y) = \{x : Ax = y\} ,$$

so that $P(A, y)$ is the set of values in S_n that map to y when multiplied by A .

- b. If r is the rank of $n \times n$ matrix A and $y \in R(A)$, prove that $|P(A, y)| = 2^{n-r}$.

Let $0 \leq m \leq n$, and suppose we partition the set S_n into blocks of consecutive numbers, where the i th block consists of the 2^m numbers $i2^m, i2^m + 1, i2^m + 2, \dots, (i+1)2^m - 1$. For any subset $S \subseteq S_n$, define $B(S, m)$ to be the set of size- 2^m blocks of S_n containing some element of S . As an example, when $n = 3, m = 1$, and $S = \{1, 4, 5\}$, then $B(S, m)$ consists of blocks 0 (since 1 is in the 0th block) and 2 (since both 4 and 5 are in block 2).

- c. Let r be the rank of the lower left $(n - m) \times m$ submatrix of A , that is, the matrix formed by taking the intersection of the bottom $n - m$ rows and the leftmost m columns of A . Let S be any size- 2^m block of S_n , and let $S' = \{y : y = Ax \text{ for some } x \in S\}$. Prove that $|B(S', m)| = 2^r$ and that for each block in $B(S', m)$, exactly 2^{m-r} numbers in S map to that block.

Because multiplying the zero vector by any matrix yields a zero vector, the set of permutations of S_n defined by multiplying by $n \times n$ 0-1 matrices with full rank over $GF(2)$ cannot include all permutations of S_n . Let us extend the class of permutations defined by matrix-vector multiplication to include an additive term, so that $x \in S_n$ maps to $Ax + c$, where c is an n -bit vector and addition is performed over $GF(2)$. For example, when

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

and

$$c = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

we get the following permutation $\pi_{A,c}$: $\pi_{A,c}(0) = 2$, $\pi_{A,c}(1) = 1$, $\pi_{A,c}(2) = 0$, $\pi_{A,c}(3) = 3$. We call any permutation that maps $x \in S_n$ to $Ax + c$, for some $n \times n$ 0-1 matrix A with full rank and some n -bit vector c , a **linear permutation**.

- d.* Use a counting argument to show that the number of linear permutations of S_n is much less than the number of permutations of S_n .
- e.* Give an example of a value of n and a permutation of S_n that cannot be achieved by any linear permutation. (*Hint:* For a given permutation, think about how multiplying a matrix by a unit vector relates to the columns of the matrix.)

Appendix notes

Linear-algebra textbooks provide plenty of background information on matrices. The books by Strang [323, 324] are particularly good.

Bibliography

- [1] Milton Abramowitz and Irene A. Stegun, editors. *Handbook of Mathematical Functions*. Dover, 1965.
- [2] G. M. Adel'son-Vel'skiĭ and E. M. Landis. An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3(5):1259–1263, 1962.
- [3] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [4] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004.
- [5] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
- [6] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [7] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [8] Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin, and Robert E. Tarjan. Faster algorithms for the shortest path problem. *Journal of the ACM*, 37(2):213–223, 1990.
- [9] Ravindra K. Ahuja and James B. Orlin. A fast and simple algorithm for the maximum flow problem. *Operations Research*, 37(5):748–759, 1989.
- [10] Ravindra K. Ahuja, James B. Orlin, and Robert E. Tarjan. Improved time bounds for the maximum flow problem. *SIAM Journal on Computing*, 18(5):939–954, 1989.
- [11] Miklós Ajtai, Nimrod Megiddo, and Orli Waarts. Improved algorithms and analysis for secretary problems and generalizations. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 473–482, 1995.
- [12] Selim G. Akl. *The Design and Analysis of Parallel Algorithms*. Prentice Hall, 1989.
- [13] Mohamad Akra and Louay Bazzi. On the solution of linear recurrence equations. *Computational Optimization and Applications*, 10(2):195–210, 1998.
- [14] Noga Alon. Generating pseudo-random permutations and maximum flow algorithms. *Information Processing Letters*, 35:201–204, 1990.

- [15] Arne Andersson. Balanced search trees made simple. In *Proceedings of the Third Workshop on Algorithms and Data Structures*, volume 709 of *Lecture Notes in Computer Science*, pages 60–71. Springer, 1993.
- [16] Arne Andersson. Faster deterministic sorting and searching in linear space. In *Proceedings of the 37th Annual Symposium on Foundations of Computer Science*, pages 135–141, 1996.
- [17] Arne Andersson, Torben Hagerup, Stefan Nilsson, and Rajeev Raman. Sorting in linear time? *Journal of Computer and System Sciences*, 57:74–93, 1998.
- [18] Tom M. Apostol. *Calculus*, volume 1. Blaisdell Publishing Company, second edition, 1967.
- [19] Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *Proceedings of the 10th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 119–129, 1998.
- [20] Sanjeev Arora. *Probabilistic checking of proofs and the hardness of approximation problems*. PhD thesis, University of California, Berkeley, 1994.
- [21] Sanjeev Arora. The approximability of NP-hard problems. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pages 337–348, 1998.
- [22] Sanjeev Arora. Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems. *Journal of the ACM*, 45(5):753–782, 1998.
- [23] Sanjeev Arora and Carsten Lund. Hardness of approximations. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 399–446. PWS Publishing Company, 1997.
- [24] Javed A. Aslam. A simple bound on the expected height of a randomly built binary search tree. Technical Report TR2001-387, Dartmouth College Department of Computer Science, 2001.
- [25] Mikhail J. Atallah, editor. *Algorithms and Theory of Computation Handbook*. CRC Press, 1999.
- [26] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, and M. Protasi. *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*. Springer, 1999.
- [27] Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. *ACM Transactions on Graphics*, 26(3), article 10, 2007.
- [28] Sara Baase and Alan Van Gelder. *Computer Algorithms: Introduction to Design and Analysis*. Addison-Wesley, third edition, 2000.
- [29] Eric Bach. Private communication, 1989.
- [30] Eric Bach. Number-theoretic algorithms. In *Annual Review of Computer Science*, volume 4, pages 119–172. Annual Reviews, Inc., 1990.
- [31] Eric Bach and Jeffrey Shallit. *Algorithmic Number Theory—Volume I: Efficient Algorithms*. The MIT Press, 1996.
- [32] David H. Bailey, King Lee, and Horst D. Simon. Using Strassen’s algorithm to accelerate the solution of linear systems. *The Journal of Supercomputing*, 4(4):357–371, 1990.

- [33] Surender Baswana, Ramesh Hariharan, and Sandeep Sen. Improved decremental algorithms for maintaining transitive closure and all-pairs shortest paths. *Journal of Algorithms*, 62(2):74–92, 2007.
- [34] R. Bayer. Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4):290–306, 1972.
- [35] R. Bayer and E. M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.
- [36] Pierre Beauchemin, Gilles Brassard, Claude Crépeau, Claude Goutier, and Carl Pomerance. The generation of random numbers that are probably prime. *Journal of Cryptology*, 1(1):53–64, 1988.
- [37] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [38] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [39] Michael Ben-Or. Lower bounds for algebraic computation trees. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, pages 80–86, 1983.
- [40] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious B-trees. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*, pages 399–409, 2000.
- [41] Samuel W. Bent and John W. John. Finding the median requires $2n$ comparisons. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, pages 213–216, 1985.
- [42] Jon L. Bentley. *Writing Efficient Programs*. Prentice Hall, 1982.
- [43] Jon L. Bentley. *Programming Pearls*. Addison-Wesley, 1986.
- [44] Jon L. Bentley, Dorothea Haken, and James B. Saxe. A general method for solving divide-and-conquer recurrences. *SIGACT News*, 12(3):36–44, 1980.
- [45] Daniel Bienstock and Benjamin McClosky. Tightening simplex mixed-integer sets with guaranteed bounds. *Optimization Online*, July 2008.
- [46] Patrick Billingsley. *Probability and Measure*. John Wiley & Sons, second edition, 1986.
- [47] Guy E. Blelloch. *Scan Primitives and Parallel Vector Models*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1989. Available as MIT Laboratory for Computer Science Technical Report MIT/LCS/TR-463.
- [48] Guy E. Blelloch. Programming parallel algorithms. *Communications of the ACM*, 39(3):85–97, 1996.
- [49] Guy E. Blelloch, Phillip B. Gibbons, and Yossi Matias. Provably efficient scheduling for languages with fine-grained parallelism. In *Proceedings of the 7th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 1–12, 1995.
- [50] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.
- [51] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. Cilk: An efficient multithreaded runtime system. *Journal of Parallel and Distributed Computing*, 37(1):55–69, 1996.

- [52] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.
- [53] Béla Bollobás. *Random Graphs*. Academic Press, 1985.
- [54] Gilles Brassard and Paul Bratley. *Fundamentals of Algorithmics*. Prentice Hall, 1996.
- [55] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.
- [56] Richard P. Brent. An improved Monte Carlo factorization algorithm. *BIT*, 20(2):176–184, 1980.
- [57] J. P. Buhler, H. W. Lenstra, Jr., and Carl Pomerance. Factoring integers with the number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 50–94. Springer, 1993.
- [58] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.
- [59] Barbara Chapman, Gabriele Jost, and Ruud van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press, 2007.
- [60] Bernard Chazelle. A minimum spanning tree algorithm with inverse-Ackermann type complexity. *Journal of the ACM*, 47(6):1028–1047, 2000.
- [61] Joseph Cheriyan and Torben Hagerup. A randomized maximum-flow algorithm. *SIAM Journal on Computing*, 24(2):203–226, 1995.
- [62] Joseph Cheriyan and S. N. Maheshwari. Analysis of preflow push algorithms for maximum network flow. *SIAM Journal on Computing*, 18(6):1057–1086, 1989.
- [63] Boris V. Cherkassky and Andrew V. Goldberg. On implementing the push-relabel method for the maximum flow problem. *Algorithmica*, 19(4):390–410, 1997.
- [64] Boris V. Cherkassky, Andrew V. Goldberg, and Tomasz Radzik. Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, 73(2):129–174, 1996.
- [65] Boris V. Cherkassky, Andrew V. Goldberg, and Craig Silverstein. Buckets, heaps, lists and monotone priority queues. *SIAM Journal on Computing*, 28(4):1326–1346, 1999.
- [66] H. Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.
- [67] Kai Lai Chung. *Elementary Probability Theory with Stochastic Processes*. Springer, 1974.
- [68] V. Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979.
- [69] V. Chvátal. *Linear Programming*. W. H. Freeman and Company, 1983.
- [70] V. Chvátal, D. A. Klarner, and D. E. Knuth. Selected combinatorial research problems. Technical Report STAN-CS-72-292, Computer Science Department, Stanford University, 1972.
- [71] Cilk Arts, Inc., Burlington, Massachusetts. *Cilk++ Programmer’s Guide*, 2008. Available at <http://www.cilk.com/archive/docs/cilk1guide>.

- [72] Alan Cobham. The intrinsic computational difficulty of functions. In *Proceedings of the 1964 Congress for Logic, Methodology, and the Philosophy of Science*, pages 24–30. North-Holland, 1964.
- [73] H. Cohen and H. W. Lenstra, Jr. Primality testing and Jacobi sums. *Mathematics of Computation*, 42(165):297–330, 1984.
- [74] Douglas Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.
- [75] Stephen Cook. The complexity of theorem proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [76] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [77] Don Coppersmith. Modifications to the number field sieve. *Journal of Cryptology*, 6(3):169–180, 1993.
- [78] Don Coppersmith and Shmuel Winograd. Matrix multiplication via arithmetic progression. *Journal of Symbolic Computation*, 9(3):251–280, 1990.
- [79] Thomas H. Cormen, Thomas Sundquist, and Leonard F. Wisniewski. Asymptotically tight bounds for performing BMMC permutations on parallel disk systems. *SIAM Journal on Computing*, 28(1):105–136, 1998.
- [80] Don Dailey and Charles E. Leiserson. Using Cilk to write multiprocessor chess programs. In H. J. van den Herik and B. Monien, editors, *Advances in Computer Games*, volume 9, pages 25–52. University of Maastricht, Netherlands, 2001.
- [81] Paolo D’Alberto and Alexandru Nicolau. Adaptive Strassen’s matrix multiplication. In *Proceedings of the 21st Annual International Conference on Supercomputing*, pages 284–292, June 2007.
- [82] Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani. *Algorithms*. McGraw-Hill, 2008.
- [83] Roman Dementiev, Lutz Kettner, Jens Mehnert, and Peter Sanders. Engineering a sorted list data structure for 32 bit keys. In *Proceedings of the Sixth Workshop on Algorithm Engineering and Experiments and the First Workshop on Analytic Algorithmics and Combinatorics*, pages 142–151, January 2004.
- [84] Camil Demetrescu and Giuseppe F. Italiano. Fully dynamic all pairs shortest paths with real edge weights. *Journal of Computer and System Sciences*, 72(5):813–837, 2006.
- [85] Eric V. Denardo and Bennett L. Fox. Shortest-route methods: 1. Reaching, pruning, and buckets. *Operations Research*, 27(1):161–186, 1979.
- [86] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994.
- [87] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, IT-22(6):644–654, 1976.
- [88] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

- [89] E. A. Dinic. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11(5):1277–1280, 1970.
- [90] Brandon Dixon, Monika Rauch, and Robert E. Tarjan. Verification and sensitivity analysis of minimum spanning trees in linear time. *SIAM Journal on Computing*, 21(6):1184–1192, 1992.
- [91] John D. Dixon. Factorization and primality tests. *The American Mathematical Monthly*, 91(6):333–352, 1984.
- [92] Dorit Dor, Johan Håstad, Staffan Ulfberg, and Uri Zwick. On lower bounds for selecting the median. *SIAM Journal on Discrete Mathematics*, 14(3):299–311, 2001.
- [93] Dorit Dor and Uri Zwick. Selecting the median. *SIAM Journal on Computing*, 28(5):1722–1758, 1999.
- [94] Dorit Dor and Uri Zwick. Median selection requires $(2 + \epsilon)n$ comparisons. *SIAM Journal on Discrete Mathematics*, 14(3):312–325, 2001.
- [95] Alvin W. Drake. *Fundamentals of Applied Probability Theory*. McGraw-Hill, 1967.
- [96] James R. Driscoll, Harold N. Gabow, Ruth Shrairman, and Robert E. Tarjan. Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation. *Communications of the ACM*, 31(11):1343–1354, 1988.
- [97] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. Making data structures persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.
- [98] Derek L. Eager, John Zahorjan, and Edward D. Lazowska. Speedup versus efficiency in parallel systems. *IEEE Transactions on Computers*, 38(3):408–423, 1989.
- [99] Herbert Edelsbrunner. *Algorithms in Combinatorial Geometry*, volume 10 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1987.
- [100] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.
- [101] Jack Edmonds. Matroids and the greedy algorithm. *Mathematical Programming*, 1(1):127–136, 1971.
- [102] Jack Edmonds and Richard M. Karp. Theoretical improvements in the algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [103] Shimon Even. *Graph Algorithms*. Computer Science Press, 1979.
- [104] William Feller. *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, third edition, 1968.
- [105] Robert W. Floyd. Algorithm 97 (SHORTEST PATH). *Communications of the ACM*, 5(6):345, 1962.
- [106] Robert W. Floyd. Algorithm 245 (TREESORT). *Communications of the ACM*, 7(12):701, 1964.
- [107] Robert W. Floyd. Permuting information in idealized two-level storage. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 105–109. Plenum Press, 1972.

- [108] Robert W. Floyd and Ronald L. Rivest. Expected time bounds for selection. *Communications of the ACM*, 18(3):165–172, 1975.
- [109] Lestor R. Ford, Jr. and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, 1962.
- [110] Lestor R. Ford, Jr. and Selmer M. Johnson. A tournament problem. *The American Mathematical Monthly*, 66(5):387–389, 1959.
- [111] Michael L. Fredman. New bounds on the complexity of the shortest path problem. *SIAM Journal on Computing*, 5(1):83–89, 1976.
- [112] Michael L. Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.
- [113] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In *Proceedings of the Twenty First Annual ACM Symposium on Theory of Computing*, pages 345–354, 1989.
- [114] Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987.
- [115] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [116] Michael L. Fredman and Dan E. Willard. Trans-dichotomous algorithms for minimum spanning trees and shortest paths. *Journal of Computer and System Sciences*, 48(3):533–551, 1994.
- [117] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.
- [118] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the 1998 ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 212–223, 1998.
- [119] Harold N. Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 74(3–4):107–114, 2000.
- [120] Harold N. Gabow, Z. Galil, T. Spencer, and Robert E. Tarjan. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica*, 6(2):109–122, 1986.
- [121] Harold N. Gabow and Robert E. Tarjan. A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences*, 30(2):209–221, 1985.
- [122] Harold N. Gabow and Robert E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989.
- [123] Zvi Galil and Oded Margalit. All pairs shortest distances for graphs with small integer length edges. *Information and Computation*, 134(2):103–139, 1997.
- [124] Zvi Galil and Oded Margalit. All pairs shortest paths for graphs with small integer length edges. *Journal of Computer and System Sciences*, 54(2):243–254, 1997.
- [125] Zvi Galil and Kunsoo Park. Dynamic programming with convexity, concavity and sparsity. *Theoretical Computer Science*, 92(1):49–76, 1992.

- [126] Zvi Galil and Joel Seiferas. Time-space-optimal string matching. *Journal of Computer and System Sciences*, 26(3):280–294, 1983.
- [127] Igal Galperin and Ronald L. Rivest. Scapegoat trees. In *Proceedings of the 4th ACM-SIAM Symposium on Discrete Algorithms*, pages 165–174, 1993.
- [128] Michael R. Garey, R. L. Graham, and J. D. Ullman. Worst-case analysis of memory allocation algorithms. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*, pages 143–150, 1972.
- [129] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [130] Saul Gass. *Linear Programming: Methods and Applications*. International Thomson Publishing, fourth edition, 1975.
- [131] Fănică Gavril. Algorithms for minimum coloring, maximum clique, minimum covering by cliques, and maximum independent set of a chordal graph. *SIAM Journal on Computing*, 1(2):180–187, 1972.
- [132] Alan George and Joseph W-H Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice Hall, 1981.
- [133] E. N. Gilbert and E. F. Moore. Variable-length binary encodings. *Bell System Technical Journal*, 38(4):933–967, 1959.
- [134] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.
- [135] Michel X. Goemans and David P. Williamson. The primal-dual method for approximation algorithms and its application to network design problems. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 144–191. PWS Publishing Company, 1997.
- [136] Andrew V. Goldberg. *Efficient Graph Algorithms for Sequential and Parallel Computers*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1987.
- [137] Andrew V. Goldberg. Scaling algorithms for the shortest paths problem. *SIAM Journal on Computing*, 24(3):494–504, 1995.
- [138] Andrew V. Goldberg and Satish Rao. Beyond the flow decomposition barrier. *Journal of the ACM*, 45(5):783–797, 1998.
- [139] Andrew V. Goldberg, Éva Tardos, and Robert E. Tarjan. Network flow algorithms. In Bernhard Korte, László Lovász, Hans Jürgen Prömel, and Alexander Schrijver, editors, *Paths, Flows, and VLSI-Layout*, pages 101–164. Springer, 1990.
- [140] Andrew V. Goldberg and Robert E. Tarjan. A new approach to the maximum flow problem. *Journal of the ACM*, 35(4):921–940, 1988.
- [141] D. Goldfarb and M. J. Todd. Linear programming. In G. L. Nemhauser, A. H. G. Rinnooy-Kan, and M. J. Todd, editors, *Handbook in Operations Research and Management Science, Vol. 1, Optimization*, pages 73–170. Elsevier Science Publishers, 1989.
- [142] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.

- [143] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.
- [144] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, third edition, 1996.
- [145] G. H. Gonnet. *Handbook of Algorithms and Data Structures*. Addison-Wesley, 1984.
- [146] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Addison-Wesley, 1992.
- [147] Michael T. Goodrich and Roberto Tamassia. *Data Structures and Algorithms in Java*. John Wiley & Sons, 1998.
- [148] Michael T. Goodrich and Roberto Tamassia. *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons, 2001.
- [149] Ronald L. Graham. Bounds for certain multiprocessor anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.
- [150] Ronald L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4):132–133, 1972.
- [151] Ronald L. Graham and Pavol Hell. On the history of the minimum spanning tree problem. *Annals of the History of Computing*, 7(1):43–57, 1985.
- [152] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics*. Addison-Wesley, second edition, 1994.
- [153] David Gries. *The Science of Programming*. Springer, 1981.
- [154] M. Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Springer, 1988.
- [155] Leo J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science*, pages 8–21, 1978.
- [156] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
- [157] H. Halberstam and R. E. Ingram, editors. *The Mathematical Papers of Sir William Rowan Hamilton*, volume III (Algebra). Cambridge University Press, 1967.
- [158] Yijie Han. Improved fast integer sorting in linear space. In *Proceedings of the 12th ACM-SIAM Symposium on Discrete Algorithms*, pages 793–796, 2001.
- [159] Yijie Han. An $O(n^3(\log \log n / \log n)^{5/4})$ time algorithm for all pairs shortest path. *Algorithmica*, 51(4):428–434, 2008.
- [160] Frank Harary. *Graph Theory*. Addison-Wesley, 1969.
- [161] Gregory C. Harfst and Edward M. Reingold. A potential-based amortized analysis of the union-find data structure. *SIGACT News*, 31(3):86–95, 2000.
- [162] J. Hartmanis and R. E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, May 1965.